UNIDAD 2 · SEMANA 9 · SESIÓN 9

Implementación de autenticación con JWT

Desarrollo Web Integrado (100000ST61) · Ingeniería de Software

Spring Boot 4 · Spring Security 7 · Java 21

HEADER
algoritmo y tipo de token

PAYLOAD
usuario, rol y vencimiento

SIGNATURE
sello de autenticidad

1 · INICIO

Rompe hielo: la pulsera del concierto

La situación

Eres el portero del Estadio Nacional. Hay 40 000 personas y una sola puerta. No puedes llamar a la boletería por cada pulsera.

¿Cómo compruebas que una pulsera es auténtica y a qué zona da acceso?

Reto en equipos de 4 (5 min)

1. Diseñen la pulsera ideal: ¿qué datos lleva escritos?
2. ¿Cómo evitan que alguien la falsifique?
3. ¿Cuándo deja de ser válida?
4. Un voluntario lo presenta en 30 segundos.

Guarda tus ideas: las retomamos al ver la anatomía de un JWT.

1 · INICIO

Repaso rápido: Spring Security

1. ¿Qué diferencia hay entre autenticación y autorización?
2. ¿Qué dependencia Maven activa Spring Security en un proyecto Spring Boot?
3. ¿Para qué sirve el bean SecurityFilterChain en nuestra aplicación?

Responde en una sola frase.

Participa por voluntarios o por sorteo.

1 · INICIO

¿Por qué importa JWT en tu carrera?

Todo backend moderno lo usa

Apps móviles, SPAs en Angular y microservicios necesitan autenticar sin guardar sesiones en el servidor.

Es parte de tu proyecto final

Tu API del Avance de Proyecto 2 (semana 10) necesita login y endpoints protegidos por rol.

Habilidad muy pedida

La seguridad de APIs es un tema frecuente en entrevistas y ofertas de desarrollador backend.

Una API sin autenticación es una puerta abierta: cualquiera puede leer o modificar tus datos.

1 · INICIO

¿Qué sabemos ya? Saberes previos

1. ¿Qué es una sesión web y dónde se guarda?
2. Una API REST es stateless: ¿qué significa eso?
3. ¿Has visto «Authorization: Bearer …» en Postman o en las DevTools? ¿Qué creías que era?

Piensa en tu celular

Abres una app de delivery o de música después de varios días y sigues dentro de tu cuenta, sin escribir tu clave en cada pantalla.

¿Cómo sabe el servidor que eres tú en cada petición?

Lluvia de ideas: 3 minutos. Anota tus palabras clave.

2 · UTILIDAD

Retos de repaso: ¡piénsalo bien!

Reto 1

Un ALUMNO ya autenticado envía POST /api/notas, una ruta que solo permite al DOCENTE.

¿Qué código HTTP recibe: 401 o 403? ¿Por qué?

Reto 2

En la configuración escribes hasRole("DOCENTE").

¿Con qué nombre exacto debe existir la autoridad del usuario en Spring Security?

Reto 3

Declaras primero una regla general: /api/notas/** para cualquier usuario autenticado. Después, una específica: POST /api/notas solo para DOCENTE.

¿Se aplica la regla específica?

2 · UTILIDAD

Logro de la sesión

Logro de la unidad (sílabo)

Al finalizar la unidad, el estudiante construye aplicaciones RESTful con conexión a bases de datos relacionales mediante Spring Boot, considerando políticas de seguridad.

Logro de hoy

Al finalizar la sesión, el estudiante implementa autenticación con JWT en una API REST con Spring Boot: genera y valida tokens firmados, configura un filtro stateless y protege endpoints por rol.

Al terminar sabrás…

- Explicar la anatomía de un JWT: header, payload y firma
- Generar y validar tokens con JJWT
- Crear el login y el filtro de autenticación
- Proteger endpoints por rol y probarlos en Postman

En tu vida profesional: podrás asegurar las APIs de un comercio electrónico, un servicio financiero o un SaaS.

2 · UTILIDAD

Utilidad e importancia de JWT

Escala fácil

El servidor no guarda sesiones: cualquier instancia puede validar el token con la clave.

Un token, muchos clientes

Angular, apps móviles y otros servicios usan el mismo mecanismo de autenticación.

Seguridad verificable

La firma impide alterar el contenido y la expiración limita el riesgo si se roba.

Estándar abierto

Definido en el RFC 7519; hay librerías para Java, JavaScript, Python y más.

2 · UTILIDAD

JWT en el mundo real

Iniciar sesión con Google
El estándar OpenID Connect entrega un ID token en formato JWT.

Apps móviles y Angular
El front guarda el token y lo envía en Authorization: Bearer (Unidad 3).

Microservicios y API Gateway
Un servicio emite el token; los demás lo validan con la firma.

GitHub Apps
Se autentican firmando un JWT con su clave privada.

```
GET /api/notas/mis-notas HTTP/1.1
Host: notasapp.utp.pe
Authorization: Bearer eyJhbGciOiJI…
    .eyJzdWIiOiJhbHVtbm8x…
    .hIXP6Qt0IDlg…
```

Así viaja el token en cada petición protegida.

3 · TRANSFORMACIÓN

Sesión en servidor vs. token

Sesión (stateful)

- El servidor guarda la sesión en memoria o base de datos.
- El cliente envía una cookie (JSESSIONID).
- Para escalar hay que compartir sesiones entre servidores.
- Ideal para páginas HTML tradicionales.

Token JWT (stateless)

- El servidor no guarda nada: el token trae la identidad.
- El cliente envía Authorization: Bearer <token>.
- Cualquier instancia valida el token con la firma.
- Ideal para APIs REST, móviles y SPAs.

Hoy usamos la columna derecha. Costo: revocar un token antes de que venza es más difícil que cerrar una sesión.

3 · TRANSFORMACIÓN

¿Qué es un JSON Web Token?

Un estándar (RFC 7519) para transmitir información entre dos partes como un objeto JSON firmado digitalmente.

Compacto

Cabe en un header HTTP y viaja rápido.

Autocontenido

Lleva usuario, rol y vencimiento dentro del token.

Verificable

La firma digital delata cualquier alteración.

Firmado ≠ cifrado: cualquiera puede LEER el payload, pero nadie puede MODIFICARLO sin que se note.

3 · TRANSFORMACIÓN

Anatomía de un JWT

eyJhbGciOiJI… . eyJzdWIiOiJhbHVt… . hIXP6Qt0IDlg…

HEADER

¿Cómo se firmó?

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Tipo de pulsera y forma de verificarla.

PAYLOAD

¿Quién eres y hasta cuándo?

```
{
  "sub": "alumno1",
  "rol": "ROLE_ALUMNO",
  "iat": 1788000000,
  "exp": 1788003600
}
```

Datos y zona escritos en la pulsera.

SIGNATURE

¿Es auténtico?

```
HMAC-SHA256(
  base64url(header) + "." +
  base64url(payload),
  clave secreta
)
```

El holograma que no se puede copiar.

3 · TRANSFORMACIÓN

Claims: los datos dentro del token

Claim	Significado	Ejemplo
sub	Subject: a quién pertenece el token	alumno1
iat	Issued at: fecha de emisión	1788000000
exp	Expiration: fecha de vencimiento	1788003600
iss	Issuer: quién emitió el token	notasapp.utp.pe
rol	Claim propio (privado) de nuestra app	ROLE_ALUMNO

Las fechas van en segundos desde 1970 (hora Unix).

Nunca pongas en el payload

- Contraseñas
- DNI, tarjetas o datos bancarios
- Cualquier dato personal sensible

El payload solo está codificado (Base64URL), no cifrado.

3 · TRANSFORMACIÓN

La firma: lo que lo hace confiable

1. El servidor firma el token al crearlo.
2. El cliente lo guarda y lo envía en cada petición.
3. El servidor recalcula la firma y la compara.
4. Si el payload cambió, no coincide: token rechazado.

Si alguien cambia rol=ALUMNO por DOCENTE → SignatureException

HS256 · simétrico

Una misma clave firma y verifica (mínimo 256 bits). Ideal para una sola aplicación, como la de hoy.

RS256 · asimétrico

La clave privada firma y la clave pública verifica. Ideal para microservicios o terceros que solo deben verificar.

3 · TRANSFORMACIÓN

Flujo completo de autenticación

A · Obtener el token (una sola vez)

1 POST /api/auth/login
usuario y clave en JSON

→

2 AuthenticationManager
valida credenciales con BCrypt

→

3 JwtService
genera el token firmado y responde 200

B · Usar el token (en cada petición)

4 Authorization: Bearer
el cliente envía el token

→

5 JwtAuthenticationFilter
valida firma y vencimiento

→

6 Controller
responde 200, o 401 / 403

3 · TRANSFORMACIÓN

¿Dónde encaja en Spring Security?

Request HTTP
↓
JwtAuthenticationFilter (nuestro)
↓
UsernamePassword AuthenticationFilter
↓
ExceptionTranslation Filter
↓
AuthorizationFilter (reglas hasRole)
↓
Controller

csrf disable
API stateless sin cookies de sesión: CSRF no aplica.

STATELESS
Spring no crea ni usa HttpSession.

addFilterBefore
Inserta el filtro JWT antes del filtro de formulario.

SecurityContext
Si queda un Authentication, las reglas ven un usuario autenticado.

Sin Authentication → 401 · Con Authentication pero sin rol → 403

3 · TRANSFORMACIÓN

Dependencias y configuración

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.13.0</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.13.0</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.13.0</version>
  <scope>runtime</scope>
</dependency>
```

application.properties

```
# clave en Base64 (mínimo 32 bytes)
jwt.secret=${JWT_SECRET:dXRwLXNlc2lvbjktand0LWNsYXZlLWRlbW8tMjU2Yml0cy0yMDI2}
# vigencia del token: 1 hora
jwt.expiration-ms=3600000
```

- Clave larga y aleatoria: mínimo 256 bits.
- En producción: variable de entorno JWT_SECRET.
- Las tres librerías JJWT, misma versión.

3 · TRANSFORMACIÓN

Ejemplo 1: generar el token

```
@Service
public class JwtService {
  private final SecretKey key;
  private final long expirationMs;

  public JwtService(@Value("${jwt.secret}") String secret,
      @Value("${jwt.expiration-ms}") long expirationMs) {
    this.key = Keys.hmacShaKeyFor(
        Decoders.BASE64.decode(secret));
    this.expirationMs = expirationMs;
  }
  public String generarToken(UserDetails user) {
    Date ahora = new Date();
    String rol = user.getAuthorities().stream()
        .findFirst().get().getAuthority();
    return Jwts.builder()
        .subject(user.getUsername())
        .claim("rol", rol)
        .issuedAt(ahora)
        .expiration(new Date(ahora.getTime() + expirationMs))
        .signWith(key)
        .compact();
  }
}
```

1. **hmacShaKeyFor**
   Convierte la clave Base64 en una SecretKey para HMAC.
2. **.subject / .claim**
   Qué va en el payload: usuario y rol.
3. **.expiration**
   Vencimiento = ahora + vigencia configurada.
4. **.signWith(key)**
   Firma; elige HS256 según el tamaño de la clave.
5. **.compact()**
   Arma el texto header.payload.signature.

3 · TRANSFORMACIÓN

Ejemplo 1: validar el token

```
// dentro de JwtService

public Claims extraerClaims(String token) {
  return Jwts.parser()
      .verifyWith(key)           // verifica la firma
      .build()
      .parseSignedClaims(token)  // y el vencimiento
      .getPayload();
}

public String extraerUsername(String token) {
  return extraerClaims(token).getSubject();
}
```

Excepción	¿Qué significa?
ExpiredJwtException	El token ya venció
SignatureException	Firma no coincide: token alterado
MalformedJwtException	El formato no es un JWT

Todas heredan de JwtException.

Si ves Jwts.parserBuilder() en un tutorial antiguo, esa API ya es obsoleta.

3 · TRANSFORMACIÓN

Ejemplo 2: endpoint de login

```
public record LoginRequest(String username, String password) {}
public record LoginResponse(String token, String tipo) {}

@RestController
@RequestMapping("/api/auth")
public class AuthController {

  private final AuthenticationManager authManager;
  private final JwtService jwtService;

  // constructor con ambos parámetros …

  @PostMapping("/login")
  public ResponseEntity<LoginResponse> login(
      @RequestBody LoginRequest req) {

    Authentication auth = authManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            req.username(), req.password()));

    UserDetails user = (UserDetails) auth.getPrincipal();
    String token = jwtService.generarToken(user);
    return ResponseEntity.ok(new LoginResponse(token, "Bearer"));
  }
}
```

Petición y respuesta

```
POST /api/auth/login

{
 "username": "alumno1",
 "password": "clave123"
}

→ 200 OK
{
 "token": "eyJhbGci…",
 "tipo": "Bearer"
}
```

authenticate() usa el UserDetailsService y BCrypt de la Sesión 8.

3 · TRANSFORMACIÓN

Ejemplo 3: el filtro JWT

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
  private final JwtService jwt;
  private final UserDetailsService uds;
  // constructor con ambos parámetros …
  @Override
  protected void doFilterInternal(HttpServletRequest req,
      HttpServletResponse res, FilterChain chain)
      throws ServletException, IOException {
    String header = req.getHeader("Authorization");
    if (header != null && header.startsWith("Bearer ")) {
      try {
        String token = header.substring(7);
        String username = jwt.extraerUsername(token);
        UserDetails ud = uds.loadUserByUsername(username);
        var auth = new UsernamePasswordAuthenticationToken(
            ud, null, ud.getAuthorities());
        SecurityContextHolder.getContext().setAuthentication(auth);
      } catch (JwtException | IllegalArgumentException e) {
        // token inválido o vencido: no se autentica
      }
    }
    chain.doFilter(req, res);
  }
}
```

1. **Leer el header**
   Authorization: Bearer <token>.
2. **substring(7)**
   Quita «Bearer » y deja solo el token.
3. **extraerUsername**
   Verifica firma y vencimiento.
4. **Authentication**
   Se crea con usuario y roles del UserDetails.
5. **SecurityContext**
   Guarda quién es el usuario en esta petición.
6. **chain.doFilter**
   Siempre se continúa la cadena.

3 · TRANSFORMACIÓN

Reglas de seguridad (stateless)

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

  @Bean
  public SecurityFilterChain filterChain(HttpSecurity http,
      JwtAuthenticationFilter jwtFilter) throws Exception {
    http
      .csrf(csrf -> csrf.disable())
      .sessionManagement(s -> s.sessionCreationPolicy(
          SessionCreationPolicy.STATELESS))
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/**").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/notas/**")
            .hasAnyRole("ALUMNO", "DOCENTE")
        .requestMatchers("/api/notas/**").hasRole("DOCENTE")
        .anyRequest().authenticated())
      .exceptionHandling(e -> e.authenticationEntryPoint(
          new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
      .addFilterBefore(jwtFilter,
          UsernamePasswordAuthenticationFilter.class);
    return http.build();
  }
}
```

- **El orden importa:** primero lo específico, al final anyRequest().
- **Entry point 401:** sin token válido, la respuesta es 401, no 403.
- **Solo lambdas:** en Spring Security 7 ya no existen and() ni antMatchers().

3 · TRANSFORMACIÓN

Beans de apoyo y error típico

```
@Configuration
public class ApplicationConfig {

  @Bean
  public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
  }

  @Bean
  public UserDetailsService userDetailsService(PasswordEncoder enc) {
    UserDetails docente = User.withUsername("docente1")
        .password(enc.encode("clave123")).roles("DOCENTE").build();
    UserDetails alumno = User.withUsername("alumno1")
        .password(enc.encode("clave123")).roles("ALUMNO").build();
    return new InMemoryUserDetailsManager(docente, alumno);
  }

  @Bean
  public AuthenticationManager authenticationManager(
      AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
  }
}
```

Error típico

Dependencia circular: la configuración necesita el filtro y el filtro necesita un bean de esa misma configuración.

Solución: separar los beans de apoyo en otra clase.

3 · TRANSFORMACIÓN

Prueba en Postman

#	Petición	Esperado
1	POST /api/auth/login (alumno1 / clave123)	200 + token
2	GET /api/notas sin token	401
3	GET /api/notas con Bearer token de ALUMNO	200
4	POST /api/notas con token de ALUMNO	403
5	GET con token alterado o vencido	401

Tips en Postman

- Login: Body → raw → JSON (no Params).
- Token: Authorization → Bearer Token.
- Pega solo el token, sin la palabra Bearer.

3 · TRANSFORMACIÓN

Buenas prácticas y errores comunes

Haz

- Expiración corta (15–60 min).
- Clave secreta en variable de entorno.
- HTTPS siempre.
- Refresh token para renovar la sesión.
- Validar firma y vencimiento.

Evita

- Datos sensibles en el payload.
- Subir la clave de prueba a GitHub.
- Tokens que nunca vencen.
- Confiar en un rol sin verificar la firma.
- Guardar el token sin conocer el riesgo XSS.

401 = no sé quién eres (token ausente, inválido o vencido). **403** = sé quién eres, pero no tienes permiso.

3 · TRANSFORMACIÓN

Dos caminos en Spring

Camino A · JJWT + filtro propio

Lo que hicimos hoy.

- Entiendes cada paso del proceso.
- Control total del token y de los claims.
- Ideal para aprender y para APIs propias.

Camino B · OAuth2 Resource Server

Soporte nativo de Spring Security.

- Valida JWT emitidos por un servidor de autorización.
- Muy usado con Keycloak, Auth0, Okta o Cognito.
- Menos código propio: lo configuras con oauth2ResourceServer.

Reto para tu proyecto: access token corto (15 min) + refresh token largo para pedir uno nuevo sin escribir la clave otra vez.

4 · PRÁCTICA

Práctica: Registro de Notas con JWT

Escenario: La facultad quiere que los docentes registren notas y que cada alumno consulte solo las suyas. Ya tienes usuarios y roles (Sesión 8): ahora agrega JWT.

Inicial 10 min

Descifra un token y escribe un test JUnit.

Usa: @Value y @Service (S2), JUnit (S3).

Intermedio 12 min

Login con validación y manejo de errores.

Usa: @RestController, ResponseEntity y códigos HTTP (S4).

Avanzado 18 min

Blindar el Registro de Notas por rol.

Usa: roles y reglas (S8), repositorio JPA (S6-S7).

4 · PRÁCTICA

Ejercicio 1 · Nivel inicial

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJkb2NlbnRlMSIsInJvbCI6IlJPTEVfRE9DRU5URSIsImlhdCI6MTc4ODAwMDAwMCw
iZXhwIjoxNzg4MDAzNjAwfQ.fRQl0HWN0IBHC1V4aAqcK0GSk1MxSbYkRevRpHB6vDc
```

Parte A · Descifra

- Pega el token en un decodificador (jwt.io).
- ¿Qué algoritmo usa? ¿Quién es el usuario y qué rol tiene?
- ¿Ya venció? ¿Pudiste leer el payload sin la clave?

Parte B · Prueba con JUnit

- Genera un token para alumno1 con tu JwtService.
- Comprueba: 3 partes, subject = alumno1 y rol = ROLE_ALUMNO.
- Reto: ¿qué excepción lanza extraerClaims con el token de arriba?

Nunca pegues tokens reales de producción en sitios web de terceros.

4 · PRÁCTICA

Ejercicio 2 · Nivel intermedio

Mejora el login para que:

1. Responda token, tipo, rol y expiraEnSegundos.
2. Usuario o clave vacíos → 400 con un mensaje.
3. Credenciales incorrectas → 401 con un mensaje.
4. Pruébalo en Postman con 3 casos: correcto, clave errónea y body vacío.

Respuesta esperada (ejemplos)

```
// 200 OK
{ "token": "eyJhbGci…",
  "tipo": "Bearer",
  "rol": "ROLE_ALUMNO",
  "expiraEnSegundos": 3600 }

// 400 Bad Request
{ "error": "Usuario y clave son obligatorios" }

// 401 Unauthorized
{ "error": "Credenciales inválidas" }
```

Pista: captura BadCredentialsException y usa ResponseEntity.status(…).body(Map.of(…)).

4 · PRÁCTICA

Ejercicio 3 · Nivel avanzado

Blinda la API para que:

- **ALUMNO:** GET /api/notas/mis-notas devuelve solo sus notas.
- **DOCENTE:** POST /api/notas registra una nota.
- **Token ausente, inválido o vencido:** 401 con JSON {"error": …}.
- **Prueba el vencimiento:** jwt.expiration-ms=60000 y espera 1 minuto.

Códigos esperados

Endpoint	ALUMNO	DOCENTE	Sin token
GET /api/notas/mis-notas	200	403	401
POST /api/notas	403	200	401
GET /api/notas	403	200	401

Adapta los nombres a tu entidad Nota de la Sesión 8.

4 · PRÁCTICA

Solución 1 · Descifrar y probar

Parte A · respuestas

- **Algoritmo:** HS256.
- **Usuario y rol:** docente1 · ROLE_DOCENTE.
- **Vencimiento:** emitido el 29/08/2026, dura 1 hora. Ya venció.
- **¿Se leyó sin clave?** Sí: está codificado, no cifrado. La clave solo verifica la firma.

Parte B · test JUnit

```
class JwtServiceTest {
  static final String SECRET =
      "dXRwLXNlc2lvbjktand0LWNsYXZlLWRlbW8tMjU2Yml0cy0yMDI2";
  // TOKEN_VENCIDO = el token del enunciado
  final JwtService jwt = new JwtService(SECRET, 3_600_000);

  @Test
  void generaTokenValido() {
    UserDetails u = User.withUsername("alumno1")
        .password("x").roles("ALUMNO").build();
    String token = jwt.generarToken(u);
    assertEquals(3, token.split("\\.").length);
    assertEquals("alumno1", jwt.extraerUsername(token));
    assertEquals("ROLE_ALUMNO",
        jwt.extraerClaims(token).get("rol", String.class));
  }
  @Test
  void tokenVencidoLanzaExcepcion() {
    assertThrows(ExpiredJwtException.class,
        () -> jwt.extraerClaims(TOKEN_VENCIDO));
  }
}
```


4 · PRÁCTICA

Solución 2 · Login con errores

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequest req) {

  if (req.username() == null || req.username().isBlank()
      || req.password() == null || req.password().isBlank()) {
    return ResponseEntity.badRequest()
        .body(Map.of("error", "Usuario y clave son obligatorios"));
  }
  try {
    Authentication auth = authManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            req.username(), req.password()));
    UserDetails user = (UserDetails) auth.getPrincipal();
    String token = jwtService.generarToken(user);
    String rol = user.getAuthorities().stream()
        .findFirst().get().getAuthority();
    return ResponseEntity.ok(Map.of("token", token,
        "tipo", "Bearer", "rol", rol, "expiraEnSegundos", 3600));
  } catch (BadCredentialsException e) {
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
        .body(Map.of("error", "Credenciales inválidas"));
  }
}
```

1. **Validar primero**
   Campos vacíos → 400.
2. **try / catch**
   BadCredentialsException → 401.
3. **Map.of**
   JSON sencillo. En un proyecto real, usa un record como DTO.
4. **ResponseEntity<?>**
   Permite cuerpos distintos según el caso.

4 · PRÁCTICA

Solución 3 · Reglas y respuesta 401

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/notas/mis-notas")
        .hasRole("ALUMNO")
    .requestMatchers("/api/notas/**").hasRole("DOCENTE")
    .anyRequest().authenticated())
.exceptionHandling(e -> e.authenticationEntryPoint(
    (req, res, ex) -> {
      res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
      res.setContentType("application/json;charset=UTF-8");
      res.getWriter().write(
          "{\"error\":\"Token ausente, inválido o vencido\"}");
    }))
```

- **Orden:** la regla de mis-notas va antes de /api/notas/**.
- **Entry point:** responde 401 en JSON cuando no hay autenticación.
- **Vencimiento:** cambia jwt.expiration-ms=60000 y espera 1 minuto.

4 · PRÁCTICA

Solución 3 · Controller de notas

```
@RestController
@RequestMapping("/api/notas")
public class NotaController {

  private final NotaRepository repo;

  public NotaController(NotaRepository repo) {
    this.repo = repo;
  }

  @GetMapping("/mis-notas")
  public List<Nota> misNotas(Authentication auth) {
    // el usuario sale del token validado
    return repo.findByAlumnoUsername(auth.getName());
  }

  @PostMapping
  public Nota registrar(@RequestBody Nota nota) {
    return repo.save(nota);
  }
}
```

Verifica en Postman

- Alumno: mis-notas → 200.
- Alumno: POST /api/notas → 403.
- Docente: POST /api/notas → 200.
- Sin token → 401 con JSON.
- Token alterado o vencido → 401.

5 · CIERRE

Armemos las conclusiones juntos

1 **Escribe**

Una frase por bloque: ¿qué es?, ¿cómo se forma?, ¿cómo viaja? y ¿cómo se valida un JWT?

2 **Ordena**

En parejas, ordenen las 6 piezas del flujo: login, credenciales, token, Bearer, filtro y controller.

3 **Comparte**

Tres voluntarios leen su frase. El docente las agrupa en la pizarra.

Completa la frase

Hoy aprendí que un JWT…

Todavía me cuesta…

Lo aplicaría en mi proyecto para…

5 · CIERRE

Mini quiz: ¿qué aprendimos?

1

¿Qué parte del JWT garantiza que el contenido no fue alterado?

A) Header
B) Payload
C) Signature
D) Claim exp

2

Un cliente envía un token vencido a /api/notas. ¿Qué respuesta esperamos con nuestra configuración?

A) 200 OK
B) 401 Unauthorized
C) 403 Forbidden
D) 500 Server Error

3

¿Cuál es una mala práctica al usar JWT?

A) Usar HTTPS
B) Expiración corta
C) Clave en variable de entorno
D) Guardar la contraseña en el payload

¡Tu API ya sabe quién eres!

Gracias por tu participación hoy. Practica lo aprendido: repite el laboratorio con otro rol, cambia el tiempo de expiración e intenta alterar un token para ver cómo reacciona la firma.

Sigue asistiendo a todas las sesiones y participa en el foro del curso: tus dudas ayudan a todos.





